

User Manual: Modular Simulation Framework

1. Introduction

This framework is designed for simulating modular mechatronic systems using interconnected modules. Each module can contain differential equations, algebraic equations, or general-purpose procedural logic. Simulations are executed in discrete time steps, and each module communicates through defined input/output ports.

2. Getting Started

To begin using the framework:

- **Ensure Python and required libraries (“numpy“, „scipy“, „pint“, „matplotlib“, „networkx“, „pandas“, „graphviz“) are installed.**
- Place all module descriptions in the „**modules/**“ subfolder.
- Run simulations using the „**main.py**“ script.
- You can just copy/paste “**Final**” folder as a starting point with empty modules folder and an empty **main()**.

3. Creating Modules

A module is defined in a Python file located in the „**modules/**“ folder and must contain:

- „**MODULE_INPUTS**“ and „**MODULE_OUTPUTS**“: Lists of „**PhysicalValue(name, value, unit)**“ create input/output ports that are visible externally.
- Optional „**PARAMETERS**“, „**INITIAL_VALUES**“, „**INTERNAL_INITIAL_VALUES**“.
- Decorated functions decide function type and name all function outputs:
 - **@differential(outputs=[...])**
 - **@algebraic(outputs=[...])**
 - **@common(outputs=[...])**
 - **@internal(outputs=[...])**

- The first two arguments of each function are `t` and `dt` (in that order); regardless of whether they are used or not.
- Importantly, not all named outputs must be defined as visible output ports; some can simply be used within a module.
- See also: **Example_Implementation/modules** folder

4. Using the Module Class

Modules are instantiated as follows:

- **`Module(name, path, parameters=..., initial_values=..., dt=..., normalise_units=True)`**

This will load the description file from the path, extract all functions and properties, and prepare it for simulation. If parameters or initial values for states are given, they will override the values written in the description file.

The variable **`normalise_units`** allows the user to write prefixes such as milli, micro, etc.

The Module and Simulation classes must be imported.

See also: **Example_Implementation** folder

5. Connecting Modules in Simulation

Use the Simulation class to define your model and connect modules:

- **`sim.connect(src_module, output_name, dst_module, input_name)`**

If **`input_name`** is omitted, **`output_name`** is used as the destination input.

All the attributes are strings and refer to the names of modules and their ports.

See also: **Example_Implementation** folder

6. Running the Simulation and real time monitoring

Run the simulation with:

- **sim.run()**

To enable real-time plotting:

- **sim.run(monitor_results=True, monitor_config=plot_config)**

The full definition of `sim.run()` is:

- **def run(self, monitor_results=False, monitor_config=None, matplotlib_backend="TkAgg", save_results=False, file_name="output_results" , plt_pause=0.001)**

This allows the user to optionally save the image under a chosen name, as well as to choose a pause taken between each real-time data adjustment. A pause is needed to have the experience of the real-time monitoring; otherwise, the data is plotted too fast, but if a big pause is chosen, it can increase the execution time too much.

The **matplotlib_backend** variable should not be changed unless, because most other backends will cause the plot to crash.

The **plot_config** dictionary has a full description of the resulting plot and can be used for both plotting the results after the execution and for real-time monitoring.

For more details about **plot_config**, check chapter 7; to see an example of `plot_config` and the general implementation, check the **Example_Implementation** folder.

7. Plot Configuration

Define a dictionary with keys:

- **„groups“**: list of output signals per subplot. Each list is a subplot. Each tuple defines an output from a module.
- **„titles“, „y_labels“, „labels“**: Optional metadata per subplot.
- **save_results** allows automatic saving of the plot. The default is False.
- **file_name** defines the name under which the plot will be saved as a .png file. The default is "output_results".
- The same plot configuration can be used for both plotting and monitoring.
- See also: **Example_Implementation** folder

8. Exporting Results to Excel

Parameters:

- **filename:** Output file name (default: "simulation_export.xlsx").
- **outputs:** List of (module_name, output_name) to export. If None, exports all.
- **time_range:** (start_time, end_time) to limit time span. The default is the entire simulation time.
- **step_skip:** Export every n-th point (default: 1).
- **force_single_sheet:** If True, aligns all outputs into one sheet with repeated values as needed. Default False, which enforces a sheet per module.

Multiple Excel files with different setups can be created, as long as the method is called multiple times, and different file names are chosen to prevent overwriting.

9. Visualisation

Parameters:

- **filename:** Output name for the PNG file (default: "simulation_graph.png").
- **show_edge_labels:** Shows variable names on connection arrows.
- **splines='true':** Enables smooth, curved edges between nodes (default). A range of options for this attribute are available, which might be better suited for a graph, depending on its use.
- **ranksep='1.1'** Sets vertical spacing between levels (in inches). Higher values add more space between module layers.
- **nodesep='0.7'** Sets horizontal spacing between nodes on the same level. Increase this to reduce visual crowding for wide diagrams.
- **show_parameters:** If False, hides parameter values from node labels. The default is True.
- **show_io:** If False, hides input/output names. The default is True.
show_dt: If False, hides dt information (only shown if modules use different dt values). The default is True.

Note: All modules and CoupledBlocks are arranged in topological order to reflect data flow.

10. Advanced Concepts

- **CoupledBlocks:** Automatically detected sets of interconnected modules with algebraic loops. Handles the functions of all its modules in the same way a module would be for its set of functions.
 - The user has nothing to do with this class; it is applied automatically in the background.
- **Internal functions:** Used for local computation or broadcasting within a module.
- **Common functions:** Used for non-equation functions, i.e., loading files, processing data, generating signals, etc.
 - They are always step-wise and never a part of algebraic loops. Even though they can belong to coupled modules, their outputs can never go in a feedback direction.

11. Units and Validation

- All variables must have units. An empty string is considered as a unitless constant.
- **Note:** There is a method within the Module class for raising errors in case of a unit mismatch. This method is deactivated now because more work is required. A detailed explanation of what has to be done is commented in the code.
- Use „**normalise_units=True**“ to convert all parameters within a module to SI, i.e., $1\text{mF} = 0.001\text{F}$.

To see details about how Units and Values in the framework work, check the file **Example_Implementation/physical_value.py**.

12. Tips

- Always provide units. An empty string is considered unitless.
- Use clear variable and function names. Do not repeat module names.
- Keep module functions concise and self-contained.
- Check connections carefully to avoid missing inputs.
 - Multiple outputs should not be connected to the same input.